Manipulação de Strings e Estruturas de Dados em COBOL

Objetivos da Aula
Ao final desta aula o aluno deverá ser capaz de:

- Utilizar `INSPECT` para contar e substituir caracteres em textos
- Inicializar registros com `INITIALIZE`
- Concatenar textos usando `STRING`
- Separar textos usando `UNSTRING`
- Obter o tamanho de campos com `FUNCTION LENGTH`
- Copiar dados entre estruturas com `MOVE CORRESPONDING`
- Manipular informações textuais comuns em sistemas corporativos
- Desenvolver sistemas de tratamento de dados cadastrais

Revisão da Aula Anterior (10 min)

O que já sabemos fazer:

Comando	Função	Exemplo
IF	Tomar decisões	IF NOTA >= 60
EVALUATE	Múltiplas decisões	EVALUATE OPCAO
PERFORM	Chamar blocos	PERFORM CABECALHO
PERFORM UNTIL	Loop condicional	PERFORM UNTIL FIM
PERFORM VARYING	Loop controlado	PERFORM VARYING I

Problema atual:

```
01 WS-NOME PIC X(30) VALUE "   JOAO   ".
01 WS-SOBRENOME PIC X(30) VALUE "   SILVA   ".
01 WS-COMPLETO PIC X(60).

???  ← Como juntar "JOAO" + "SILVA" sem espaços extras?
```

> 🤔 ***Desafio de hoje:*** *Manipular textos de forma eficiente!*

1 – O Problema dos Dados Textuais (5 min)

Por que manipular textos é importante?

Em sistemas corporativos, **80% dos dados** são textos:

Tipo de Dado	Exemplo
Nome completo	"JOAO SILVA SANTOS"
Endereço	"RUA DAS FLORES, 100"
E-mail	"joao@empresa.com"
CPF/CNPJ	"123.456.789-00"
Telefone	"(11) 99999-9999"
Arquivos	Leitura e escrita de dados

Desafios comuns:

```
"   JOAO   " + "   SILVA   " = "   JOAO   SILVA   "  ← Com espaços extras!
"JOAO;SILVA;35;SP" → Separar em 4 campos
"BANANA" → Quantos "A" tem? → 3
```

Hoje aprenderemos a resolver esses problemas!

2 – Comando INSPECT (20 min)

O que é INSPECT?

Comando especializado para **analisar e modificar textos** sem precisar de loops.

Analogia: Editor de Texto

```
INSPECT = Localizar + Substituir + Contar
```

Função 1: TALLYING (Contar caracteres)

Conta quantas vezes um caractere ou palavra aparece.

Sintaxe:

```
INSPECT texto
    TALLYING contador
    FOR ALL caractere.
```

Exemplo 1 - Contando Letras:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-CONTAR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO    PIC X(30) VALUE "BANANA".
01 WS-QTD-A    PIC 99.

PROCEDURE DIVISION.
    INSPECT WS-TEXTO
        TALLYING WS-QTD-A
        FOR ALL "A".

    DISPLAY "TEXTO: " WS-TEXTO.
    DISPLAY "QUANTIDADE DE 'A': " WS-QTD-A.
    STOP RUN.
```

Saída:

```
TEXTO: BANANA
QUANTIDADE DE 'A': 3
```

Exemplo 2 - Contando Palavras:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-PALAVRAS.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO    PIC X(50) VALUE "COBOL COBOL COBOL".
01 WS-QTD      PIC 99.

PROCEDURE DIVISION.
    INSPECT WS-TEXTO
        TALLYING WS-QTD
        FOR ALL "COBOL".

    DISPLAY "TEXTO: " WS-TEXTO.
    DISPLAY "QUANTIDADE DE 'COBOL': " WS-QTD.
    STOP RUN.
```

Saída:

```
TEXTO: COBOL COBOL COBOL
QUANTIDADE DE 'COBOL': 3
```

Exemplo 3 - Contando Vogais:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-VOGAIS.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO    PIC X(30) VALUE "PROGRAMACAO COBOL".
01 WS-QTD-VOGAL PIC 99.

PROCEDURE DIVISION.
    INSPECT WS-TEXTO
        TALLYING WS-QTD-VOGAL
        FOR ALL "A"
        FOR ALL "E"
        FOR ALL "I"
        FOR ALL "O"
        FOR ALL "U".

    DISPLAY "TEXTO: " WS-TEXTO.
    DISPLAY "QUANTIDADE DE VOGAIS: " WS-QTD-VOGAL.
    STOP RUN.
```

Saída:

```
TEXTO: PROGRAMACAO COBOL
QUANTIDADE DE VOGAIS: 8
```

Função 2: REPLACING (Substituir)

Substitui caracteres ou palavras por outros.

Sintaxe:

```
INSPECT texto
    REPLACING ALL caractere
    BY novo-caractere.
```

Exemplo 1 - Substituindo Caracteres:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-SUBSTITUIR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO PIC X(20) VALUE "BANANA".

PROCEDURE DIVISION.
    DISPLAY "ANTES: " WS-TEXTO.

    INSPECT WS-TEXTO
        REPLACING ALL "A"
        BY "X".

    DISPLAY "DEPOIS: " WS-TEXTO.
    STOP RUN.
```

Saída:

```
ANTES: BANANA
DEPOIS: BXNXNX
```

Exemplo 2 - Substituindo Palavras:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-SUBST-PALAVRA.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME PIC X(30) VALUE "JOAO DA SILVA".

PROCEDURE DIVISION.
    DISPLAY "ANTES: " WS-NOME.

    INSPECT WS-NOME
        REPLACING ALL "JOAO"
        BY "MARIA".

    DISPLAY "DEPOIS: " WS-NOME.
    STOP RUN.
```

Saída:

```
ANTES: JOAO DA SILVA
DEPOIS: MARIA DA SILVA
```

Exemplo 3 - Substituindo Múltiplos:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-MULTIPLO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO PIC X(30) VALUE "AULA DE COBOL".

PROCEDURE DIVISION.
    DISPLAY "ANTES: " WS-TEXTO.

    INSPECT WS-TEXTO
        REPLACING ALL "A" BY "X"
        AND ALL " " BY "-".

    DISPLAY "DEPOIS: " WS-TEXTO.
    STOP RUN.
```

Saída:

```
ANTES: AULA DE COBOL
DEPOIS: XULX-DE-COBOL
```

INSPECT com BEFORE/AFTER

Limita a contagem/substituição a uma parte do texto.

Exemplo - Contar antes de um caractere:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INSPECT-BEFORE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO PIC X(30) VALUE "JOAO-SILVA-SANTOS".
01 WS-QTD   PIC 99.

PROCEDURE DIVISION.
    INSPECT WS-TEXTO
        TALLYING WS-QTD
        FOR ALL "A"
        BEFORE INITIAL "-".

    DISPLAY "TEXTO: " WS-TEXTO.
    DISPLAY "'A' ANTES DO PRIMEIRO '-': " WS-QTD.
    STOP RUN.
```

Saída:

```
TEXTO: JOAO-SILVA-SANTOS
'A' ANTES DO PRIMEIRO '-': 2
```

Resumo INSPECT

Função	Comando	O que faz
Contar	`INSPECT TALLYING FOR ALL`	Conta ocorrências
Substituir	`INSPECT REPLACING ALL BY`	Substitui caracteres
Contar antes	`BEFORE INITIAL`	Limita a contagem
Substituir antes	`BEFORE INITIAL`	Limita a substituição

3 – INITIALIZE (15 min)

O que é INITIALIZE?

Comando que **zera ou limpa** variáveis de uma só vez.

Analogia: Limpeza Geral

```
INITIALIZE = Limpar todo o registro de uma vez
```

Sintaxe Básica

```
INITIALIZE nome-estrutura.
```

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INIT-EXEMPLO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-CLIENTE.
   05 WS-NOME    PIC X(30) VALUE "JOAO SILVA".
   05 WS-IDADE   PIC 9(3)  VALUE 25.
   05 WS-SALARIO PIC 9(6)V99 VALUE 5000.00.
```

```
PROCEDURE DIVISION.
    DISPLAY "ANTES DO INITIALIZE:".
    DISPLAY "NOME: " WS-NOME.
    DISPLAY "IDADE: " WS-IDADE.
    DISPLAY "SALARIO: " WS-SALARIO.

    INITIALIZE WS-CLIENTE.

    DISPLAY " ".
    DISPLAY "DEPOIS DO INITIALIZE:".
    DISPLAY "NOME: " WS-NOME.
    DISPLAY "IDADE: " WS-IDADE.
    DISPLAY "SALARIO: " WS-SALARIO.

    STOP RUN.
```

Saída:

```
ANTES DO INITIALIZE:
NOME: JOAO SILVA
IDADE: 25
SALARIO: 5000.00

DEPOIS DO INITIALIZE:
NOME:
IDADE: 000
SALARIO: 000000.00
```

O que INITIALIZE faz em cada tipo:

Tipo de Campo	O que acontece
`PIC X`	Preenche com SPACES
`PIC 9`	Preenche com ZEROS
`PIC X` com VALUE	Perde o valor inicial

INITIALIZE com REPLACING

Permite personalizar o valor de preenchimento.

Exemplo - Preencher com "*":

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INIT-REPLACING.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-CLIENTE.
   05 WS-NOME    PIC X(30) VALUE "JOAO SILVA".
   05 WS-IDADE   PIC 9(3)  VALUE 25.
   05 WS-SALARIO PIC 9(6)V99 VALUE 5000.00.

PROCEDURE DIVISION.
    DISPLAY "ANTES:".
    DISPLAY "NOME: " WS-NOME.
    DISPLAY "IDADE: " WS-IDADE.
    DISPLAY "SALARIO: " WS-SALARIO.

    INITIALIZE WS-CLIENTE
        REPLACING ALPHANUMERIC DATA BY "*".

    DISPLAY " ".
    DISPLAY "DEPOIS:".
    DISPLAY "NOME: " WS-NOME.
    DISPLAY "IDADE: " WS-IDADE.
    DISPLAY "SALARIO: " WS-SALARIO.

    STOP RUN.
```

Saída:

```
ANTES:
NOME: JOAO SILVA
IDADE: 25
SALARIO: 5000.00

DEPOIS:
NOME:
IDADE: 000
SALARIO: 000000.00
```

> 💡 ***Dica**: `REPLACING` é útil para mascarar dados sensíveis!*

4 – STRING (20 min)

O que é STRING?

Comando para **concatenar (juntar)** textos.

Analogia: Cola de Texto

```
STRING = Cola + Fita adesiva para textos
```

Sintaxe Básica

```
STRING
    parte1
    parte2
    parte3
INTO destino.
```

Exemplo - Concatenação Simples:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. STRING-SIMPLES.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME      PIC X(10) VALUE "JOAO".
01 WS-SOBRENOME PIC X(10) VALUE "SILVA".
01 WS-COMPLETO  PIC X(30).

PROCEDURE DIVISION.
    STRING
        WS-NOME
        WS-SOBRENOME
    INTO WS-COMPLETO.

    DISPLAY "NOME: [" WS-COMPLETO "]".
    STOP RUN.
```

Saída:

```
NOME: [JOAO      SILVA     ]
```

> ⚠ ***Problema****: Os espaços extras foram preservados!*

DELIMITED BY - Solução

Controla até onde copiar cada parte.

Tipos de DELIMITED:

Opção	O que faz
`DELIMITED BY SPACE`	Copia até o espaço
`DELIMITED BY SIZE`	Copia o campo inteiro (padrão)
`DELIMITED BY "X"`	Copia até o caractere X
`DELIMITED BY WS-LIMITE`	Copia até o valor da variável

Exemplo com DELIMITED BY SPACE:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. STRING-DELIMITED.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME      PIC X(10) VALUE "JOAO".
01 WS-SOBRENOME PIC X(10) VALUE "SILVA".
01 WS-COMPLETO  PIC X(30).

PROCEDURE DIVISION.
    STRING
        WS-NOME DELIMITED BY SPACE
        WS-SOBRENOME DELIMITED BY SPACE
    INTO WS-COMPLETO.

    DISPLAY "NOME: [" WS-COMPLETO "]".
    STOP RUN.
```

Saída:

```
NOME: [JOAOSILVA                    ]
```

> Ainda não temos o espaço entre nome e sobrenome!

Adicionando Espaço:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. STRING-ESPACO.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME      PIC X(10) VALUE "JOAO".
01 WS-SOBRENOME PIC X(10) VALUE "SILVA".
01 WS-COMPLETO  PIC X(30).

PROCEDURE DIVISION.
    STRING
        WS-NOME DELIMITED BY SPACE
        " "
        WS-SOBRENOME DELIMITED BY SPACE
    INTO WS-COMPLETO.

    DISPLAY "NOME: [" WS-COMPLETO "]".
    STOP RUN.
```

Saída:

```
NOME: [JOAO SILVA                  ]
```

WITH POINTER - Controlando Posição

Define onde começar a escrever.

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. STRING-POINTER.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-COMPLETO  PIC X(30) VALUE "---FIM---".
01 WS-PONTEIRO  PIC 99 VALUE 4.  ← Começa na posição 4
01 WS-NOME      PIC X(10) VALUE "JOAO".

PROCEDURE DIVISION.
    STRING
        WS-NOME DELIMITED BY SPACE
    INTO WS-COMPLETO
    WITH POINTER WS-PONTEIRO.

    DISPLAY "RESULTADO: [" WS-COMPLETO "]".
    DISPLAY "PONTEIRO FINAL: " WS-PONTEIRO.
    STOP RUN.
```

Saída:

```
RESULTADO: [---JOAO FIM---]
PONTEIRO FINAL: 8
```

> 💡 ***Útil**: Para montar relatórios em posições específicas!*

ON OVERFLOW - Tratamento de Erro

Detecta quando o destino é muito pequeno.

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. STRING-OVERFLOW.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-PARTE1 PIC X(20) VALUE "TEXTO MUITO LONGO".
01 WS-PARTE2 PIC X(20) VALUE "E MAIS TEXTO".
01 WS-DESTINO PIC X(10).

PROCEDURE DIVISION.
    STRING
        WS-PARTE1 DELIMITED BY SIZE
        WS-PARTE2 DELIMITED BY SIZE
    INTO WS-DESTINO
    ON OVERFLOW
        DISPLAY "ERRO! DESTINO MUITO PEQUENO!"
    NOT ON OVERFLOW
        DISPLAY "CONCATENAÇÃO REALIZADA!"
    END-STRING.

    DISPLAY "DESTINO: [" WS-DESTINO "]".
    STOP RUN.
```

Saída:

```
ERRO! DESTINO MUITO PEQUENO!
DESTINO: [TEXTO MUIT]
```

5 – UNSTRING (20 min)

O que é UNSTRING?

Operação **inversa** do STRING: **divide** um texto em partes.

Analogia: Cortador de Pizza

```
UNSTRING = Corta um texto grande em pedaços
```

Sintaxe Básica

```
UNSTRING texto-origem
    DELIMITED BY separador
    INTO destino1
         destino2
         destino3
```

Exemplo - Separando por ";":

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNSTRING-SIMPLES.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-DADOS     PIC X(30) VALUE "JOAO;SILVA;35;SP".
01 WS-NOME      PIC X(20).
01 WS-SOBRENOME PIC X(20).
01 WS-IDADE     PIC X(3).
01 WS-CIDADE    PIC X(20).

PROCEDURE DIVISION.
    UNSTRING WS-DADOS
        DELIMITED BY ";"
        INTO WS-NOME
             WS-SOBRENOME
             WS-IDADE
             WS-CIDADE.

    DISPLAY "DADOS ORIGINAIS: " WS-DADOS.
    DISPLAY "NOME:      " WS-NOME.
    DISPLAY "SOBRENOME: " WS-SOBRENOME.
    DISPLAY "IDADE:     " WS-IDADE.
    DISPLAY "CIDADE:    " WS-CIDADE.

    STOP RUN.
```

Saída:

```
DADOS ORIGINAIS: JOAO;SILVA;35;SP
NOME:      JOAO
SOBRENOME: SILVA
IDADE:     35
CIDADE:    SP
```

COUNT IN - Contar caracteres copiados

Mostra quantos caracteres foram copiados para cada campo.

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNSTRING-COUNT.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-DADOS     PIC X(30) VALUE "JOAO;SILVA;35;SP".
01 WS-NOME      PIC X(20).
01 WS-SOBRENOME PIC X(20).
01 WS-IDADE     PIC X(3).
01 WS-CIDADE    PIC X(20).
01 WS-TAM-NOME  PIC 99.
01 WS-TAM-SOBR  PIC 99.

PROCEDURE DIVISION.
    UNSTRING WS-DADOS
        DELIMITED BY ";"
        INTO WS-NOME  COUNT IN WS-TAM-NOME
             WS-SOBRENOME COUNT IN WS-TAM-SOBR
             WS-IDADE
             WS-CIDADE.

    DISPLAY "NOME: [" WS-NOME "] TAM: " WS-TAM-NOME.
    DISPLAY "SOBRENOME: [" WS-SOBRENOME "] TAM: " WS-TAM-SOBR.
    STOP RUN.
```

Saída:

```
NOME: [JOAO] TAM: 4
SOBRENOME: [SILVA] TAM: 5
```

TALLYING IN - Contar campos separados

Conta quantos campos foram preenchidos.

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNSTRING-TALLY.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-DADOS     PIC X(30) VALUE "JOAO;SILVA;35;SP".
01 WS-CAMPO1    PIC X(20).
01 WS-CAMPO2    PIC X(20).
01 WS-CAMPO3    PIC X(20).
01 WS-CAMPO4    PIC X(20).
01 WS-CAMPOS    PIC 99.

PROCEDURE DIVISION.
    UNSTRING WS-DADOS
        DELIMITED BY ";"
        INTO WS-CAMPO1
             WS-CAMPO2
             WS-CAMPO3
             WS-CAMPO4
        TALLYING IN WS-CAMPOS.

    DISPLAY "CAMPOS ENCONTRADOS: " WS-CAMPOS.
    STOP RUN.
```

Saída:

```
CAMPOS ENCONTRADOS: 4
```

ON OVERFLOW - Mais campos que destinos

Quando há mais partes que variáveis destino.

Exemplo:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNSTRING-OVERFLOW.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-DADOS     PIC X(30) VALUE "JOAO;SILVA;35;SP".
01 WS-CAMPO1    PIC X(20).
01 WS-CAMPO2    PIC X(20).

PROCEDURE DIVISION.
    UNSTRING WS-DADOS
        DELIMITED BY ";"
```

```
        INTO WS-CAMPO1
             WS-CAMPO2
        ON OVERFLOW
            DISPLAY "MAIS DE 2 CAMPOS ENCONTRADOS!"
    END-UNSTRING.

    DISPLAY "CAMPO1: " WS-CAMPO1.
    DISPLAY "CAMPO2: " WS-CAMPO2.
    STOP RUN.
```

Saída:

```
MAIS DE 2 CAMPOS ENCONTRADOS!
CAMPO1: JOAO
CAMPO2: SILVA
```

6 – FUNCTION LENGTH (10 min)

O que é LENGTH?

Função que retorna o **tamanho** de um campo.

Analogia: Fita Métrica

```
LENGTH = Medir o comprimento do texto
```

Sintaxe

```
FUNCTION LENGTH(variavel)
```

Exemplo - Tamanho de Campos:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. LENGTH-EXEMPLO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME      PIC X(30) VALUE "JOAO SILVA".
01 WS-TAMANHO   PIC 99.

PROCEDURE DIVISION.
    MOVE FUNCTION LENGTH(WS-NOME) TO WS-TAMANHO.
    DISPLAY "NOME: [" WS-NOME "]".
    DISPLAY "TAMANHO: " WS-TAMANHO.
    STOP RUN.
```

Saída:

```
NOME: [JOAO SILVA                ]
TAMANHO: 30
```

> ⚠ ***Atenção**: LENGTH retorna o tamanho **declarado**, não o tamanho real do texto!*

Para obter o tamanho real do texto (sem espaços):

Use `INSPECT` com `TALLYING` para contar caracteres.

Exemplo - Tamanho Real:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. LENGTH-REAL.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTO    PIC X(30) VALUE "JOAO SILVA".
01 WS-TAM      PIC 99.
01 WS-CONT     PIC 99.

PROCEDURE DIVISION.
    *>> Tamanho declarado
    MOVE FUNCTION LENGTH(WS-TEXTO) TO WS-TAM.
    DISPLAY "TAMANHO DECLARADO: " WS-TAM.

    *>> Tamanho real (sem espaços)
    PERFORM VARYING WS-CONT FROM 1 BY 1
        UNTIL WS-CONT > 30
        IF WS-TEXTO(WS-CONT:1) = SPACE
            DISPLAY "PRIMEIRO ESPAÇO EM: " WS-CONT
            STOP RUN
        END-IF
    END-PERFORM.

    STOP RUN.
```

> 💡 ***Na prática:** Use `STRING` com `DELIMITED BY SIZE` para obter tamanho real.*

7 – MOVE CORRESPONDING (15 min)

O que é MOVE CORRESPONDING?

Copia automaticamente **campos com o mesmo nome** entre duas estruturas.

Analogia: Cópia Seletiva

```
MOVE CORR = Copiar apenas o que tem o mesmo nome
```

Exemplo - Sem CORRESPONDING (Manual):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOVE-MANUAL.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-ORIGEM.
   05 WS-NOME      PIC X(30) VALUE "JOAO".
   05 WS-IDADE     PIC 9(3)  VALUE 25.
   05 WS-CIDADE    PIC X(20) VALUE "BARUERI".
   05 WS-SALARIO   PIC 9(6)V99 VALUE 5000.00.

01 WS-DESTINO.
   05 WS-NOME      PIC X(30).
   05 WS-IDADE     PIC 9(3).
   05 WS-CIDADE    PIC X(20).
   05 WS-CARGO     PIC X(20).

PROCEDURE DIVISION.
    *>> Copiando manualmente
    MOVE WS-NOME      TO WS-NOME
    MOVE WS-IDADE     TO WS-IDADE
    MOVE WS-CIDADE    TO WS-CIDADE.

    DISPLAY "DESTINO:".
    DISPLAY "NOME: " WS-NOME.
    DISPLAY "IDADE: " WS-IDADE.
    DISPLAY "CIDADE: " WS-CIDADE.
    STOP RUN.
```

Exemplo - Com CORRESPONDING (Automático):

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOVE-CORR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-ORIGEM.
   05 WS-NOME      PIC X(30) VALUE "JOAO".
   05 WS-IDADE     PIC 9(3)  VALUE 25.
   05 WS-CIDADE    PIC X(20) VALUE "BARUERI".
   05 WS-SALARIO   PIC 9(6)V99 VALUE 5000.00.

01 WS-DESTINO.
   05 WS-NOME      PIC X(30).
   05 WS-IDADE     PIC 9(3).
   05 WS-CIDADE    PIC X(20).
   05 WS-CARGO     PIC X(20).

PROCEDURE DIVISION.
    *>> Copiando automático
    MOVE CORRESPONDING WS-ORIGEM
        TO WS-DESTINO.

    DISPLAY "DESTINO:".
    DISPLAY "NOME: " WS-NOME.
    DISPLAY "IDADE: " WS-IDADE.
    DISPLAY "CIDADE: " WS-CIDADE.
    DISPLAY "CARGO: " WS-CARGO.  ← Ficou vazio
    STOP RUN.
```

Saída:

```
DESTINO:
NOME: JOAO
IDADE: 25
CIDADE: BARUERI
CARGO:
```

> 💡 ***Vantagem**: Copia apenas os campos com o mesmo nome automaticamente!*

CORRESPONDING com Prefixos Diferentes:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOVE-CORR-PREFIXO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-ORIGEM.
   05 WS-CLI-NOME      PIC X(30) VALUE "JOAO".
   05 WS-CLI-IDADE     PIC 9(3)  VALUE 25.
   05 WS-CLI-CIDADE    PIC X(20) VALUE "BARUERI".

01 WS-DESTINO.
   05 WS-CLI-NOME      PIC X(30).
   05 WS-CLI-IDADE     PIC 9(3).
   05 WS-CLI-CIDADE    PIC X(20).

PROCEDURE DIVISION.
    MOVE CORRESPONDING WS-ORIGEM
        TO WS-DESTINO.

    DISPLAY "NOME: " WS-CLI-NOME.
    DISPLAY "IDADE: " WS-CLI-IDADE.
    DISPLAY "CIDADE: " WS-CLI-CIDADE.
    STOP RUN.
```

> ⚠ ***Regra**: Os nomes dos campos devem ser **exatamente iguais** (incluindo prefixos)!*

Regras do MOVE CORRESPONDING:

Situação	O que acontece
Campos com mesmo nome	Copiados
Campos com nomes diferentes	Ignorados
Campos com tamanhos diferentes	Cópia normal (com truncamento)

Exercício Guiado (10 min)

Programa - Tratamento de Dados

Crie um programa que:

1. Receba nome e sobrenome
2. Use STRING para criar nome completo
3. Use LENGTH para mostrar tamanho
4. Use UNSTRING para separar novamente

Resolução:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXERCICIO-STRING.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME      PIC X(20).
01 WS-SOBRENOME PIC X(20).
01 WS-COMPLETO  PIC X(40).
01 WS-TAMANHO   PIC 99.

PROCEDURE DIVISION.
    DISPLAY "DIGITE O NOME:".
    ACCEPT WS-NOME.
    DISPLAY "DIGITE O SOBRENOME:".
    ACCEPT WS-SOBRENOME.

    *>> CONCATENAR
    STRING
        WS-NOME DELIMITED BY SPACE
        " "
        WS-SOBRENOME DELIMITED BY SPACE
    INTO WS-COMPLETO.

    *>> MOSTRAR RESULTADO
    DISPLAY "NOME COMPLETO: [" WS-COMPLETO "]".

    *>> TAMANHO
    MOVE FUNCTION LENGTH(WS-COMPLETO) TO WS-TAMANHO.
    DISPLAY "TAMANHO DECLARADO: " WS-TAMANHO.

    STOP RUN.
```

Atividade Prática da Aula (30 min)

🎯 **Sistema de Cadastro de Clientes com Tratamento de Dados**

Objetivo:
Criar um sistema completo de cadastro que utilize todas as técnicas de manipulação de strings.

Cenário:
Uma empresa precisa cadastrar clientes e processar seus dados.

Programa Completo:

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. CADASTRO-CLIENTE-STRINGS.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       *> ESTRUTURA DE ENTRADA
       01 WS-ENTRADA.
          05 WS-NOME        PIC X(30).
          05 WS-SOBRENOME   PIC X(30).
          05 WS-END-COMPLETO PIC X(60).
          05 WS-CIDADE      PIC X(30).
          05 WS-ESTADO      PIC X(2).
          05 WS-CEP         PIC X(9).

       *> ESTRUTURA PROCESSADA
       01 WS-CLIENTE.
          05 WS-NOME-COMPLETO PIC X(60).
          05 WS-ENDEREÇO.
             10 WS-RUA        PIC X(40).
             10 WS-NUMERO     PIC X(6).
             10 WS-BAIRRO     PIC X(30).
          05 WS-CIDADE-ESTADO PIC X(40).
          05 WS-CEP           PIC X(9).

       *> ESTRUTURA DE EXIBIÇÃO
       01 WS-RELATORIO.
          05 WS-LINHA PIC X(60) VALUE

"------------------------------------------------------------".

       01 WS-AUXILIARES.
          05 WS-TAM-NOME      PIC 99.
          05 WS-TAM-ENDERECO  PIC 99.
```

```
           05 WS-I             PIC 99.
           05 WS-QTD-CAMPOS    PIC 99.
           05 WS-OPCAO         PIC X(1).

       PROCEDURE DIVISION.
       *> ======================================
       *> TÍTULO PRINCIPAL
       *> ======================================
           PERFORM CABECALHO.

           PERFORM UNTIL WS-OPCAO = "N"
               PERFORM ENTRADA-DADOS
               PERFORM PROCESSAR-DADOS
               PERFORM EXIBIR-CLIENTE
               PERFORM PERGUNTAR-CONTINUAR
           END-PERFORM.

           DISPLAY "======================================".
           DISPLAY "        PROGRAMA ENCERRADO".
           DISPLAY "======================================".

           STOP RUN.

       *> ======================================
       *> CABECALHO
       *> ======================================
       CABECALHO.
           DISPLAY "======================================".
           DISPLAY "   SISTEMA DE CADASTRO DE CLIENTES".
           DISPLAY "   COM TRATAMENTO DE STRINGS".
           DISPLAY "======================================".
           DISPLAY " ".
           EXIT.

       *> ======================================
       *> ENTRADA DE DADOS
       *> ======================================
       ENTRADA-DADOS.
           INITIALIZE WS-ENTRADA.
           INITIALIZE WS-CLIENTE.

           DISPLAY "DADOS DO CLIENTE".
           DISPLAY WS-LINHA.
           DISPLAY "NOME:".
           ACCEPT WS-NOME.
```

```
           DISPLAY "SOBRENOME:".
           ACCEPT WS-SOBRENOME.

           DISPLAY "ENDEREÇO COMPLETO (RUA, NÚMERO, BAIRRO):".
           ACCEPT WS-END-COMPLETO.

           DISPLAY "CIDADE:".
           ACCEPT WS-CIDADE.

           DISPLAY "ESTADO (UF):".
           ACCEPT WS-ESTADO.

           DISPLAY "CEP:".
           ACCEPT WS-CEP.

           DISPLAY " ".
           EXIT.

       *> ======================================
       *> PROCESSAR DADOS
       *> ======================================
       PROCESSAR-DADOS.
           *>> 1. CONCATENAR NOME COMPLETO
           STRING
               WS-NOME DELIMITED BY SPACE
               " "
               WS-SOBRENOME DELIMITED BY SPACE
           INTO WS-NOME-COMPLETO.

           *>> 2. SEPARAR ENDEREÇO (RUA, NÚMERO, BAIRRO)
           UNSTRING WS-END-COMPLETO
               DELIMITED BY ","
               INTO WS-RUA
                    WS-NUMERO
                    WS-BAIRRO
               TALLYING IN WS-QTD-CAMPOS
           ON OVERFLOW
               DISPLAY "   * ENDEREÇO COM MAIS DE 3 PARTES"
           END-UNSTRING.

           *>> 3. JUNTAR CIDADE + ESTADO
           STRING
               WS-CIDADE DELIMITED BY SIZE
               "/"
               WS-ESTADO DELIMITED BY SIZE
           INTO WS-CIDADE-ESTADO.
```

```
        *>> 4. LIMPEZA DE ESPAÇOS (INSPECT)
        INSPECT WS-RUA REPLACING ALL SPACE BY SPACE.

        *>> 5. CONTAGEM DE CARACTERES
        MOVE FUNCTION LENGTH(WS-NOME-COMPLETO)
            TO WS-TAM-NOME.

        DISPLAY "   * TAMANHO NOME: " WS-TAM-NOME.
        DISPLAY "   * CAMPOS ENDEREÇO: " WS-QTD-CAMPOS.

        EXIT.

    *> ======================================
    *> EXIBIR CLIENTE
    *> ======================================
    EXIBIR-CLIENTE.
        DISPLAY " ".
        DISPLAY "======================================".
        DISPLAY "         FICHA DO CLIENTE".
        DISPLAY "======================================".
        DISPLAY " ".

        DISPLAY "NOME COMPLETO:".
        DISPLAY WS-LINHA.
        DISPLAY WS-NOME-COMPLETO.
        DISPLAY " ".

        DISPLAY "ENDEREÇO:".
        DISPLAY WS-LINHA.
        DISPLAY "RUA    : " WS-RUA.
        DISPLAY "NÚMERO : " WS-NUMERO.
        DISPLAY "BAIRRO : " WS-BAIRRO.
        DISPLAY "CIDADE : " WS-CIDADE-ESTADO.
        DISPLAY "CEP    : " WS-CEP.
        DISPLAY " ".

        DISPLAY "======================================".
        EXIT.

    *> ======================================
    *> PERGUNTAR CONTINUAR
    *> ======================================
    PERGUNTAR-CONTINUAR.
        DISPLAY " ".
        DISPLAY "DESEJA CADASTRAR OUTRO CLIENTE? (S/N)".
```

```
           ACCEPT WS-OPCAO.
           DISPLAY " ".
           EXIT.
```

Saída Esperada:

```
========================================
   SISTEMA DE CADASTRO DE CLIENTES
   COM TRATAMENTO DE STRINGS
========================================

DADOS DO CLIENTE
------------------------------------------------------------
NOME:
JOAO
SOBRENOME:
SILVA
ENDEREÇO COMPLETO (RUA, NÚMERO, BAIRRO):
RUA DAS FLORES, 100, CENTRO
CIDADE:
SAO PAULO
ESTADO (UF):
SP
CEP:
01234-567

   * TAMANHO NOME: 11
   * CAMPOS ENDEREÇO: 3

========================================
         FICHA DO CLIENTE
========================================

NOME COMPLETO:
------------------------------------------------------------
JOAO SILVA

ENDEREÇO:
------------------------------------------------------------
RUA    : RUA DAS FLORES
NÚMERO : 100
BAIRRO : CENTRO
CIDADE : SAO PAULO/SP
CEP    : 01234-567

========================================
```

```
DESEJA CADASTRAR OUTRO CLIENTE? (S/N)
N

======================================
        PROGRAMA ENCERRADO
======================================
```

Resumo dos Comandos de Manipulação de Strings

Comando	Função	Exemplo
`INSPECT TALLYING`	Contar caracteres	`INSPECT TEXTO TALLYING QTD FOR ALL "A"`
`INSPECT REPLACING`	Substituir caracteres	`INSPECT TEXTO REPLACING ALL "A" BY "X"`
`INITIALIZE`	Limpar estruturas	`INITIALIZE WS-CLIENTE`
`STRING`	Concatenar textos	`STRING NOME " " SOBRENOME INTO COMPLETO`
`UNSTRING`	Dividir textos	`UNSTRING DADOS DELIMITED BY ";" INTO CAMPO1`
`FUNCTION LENGTH`	Tamanho do campo	`FUNCTION LENGTH(WS-NOME)`
`MOVE CORRESPONDING`	Copiar campos com mesmo nome	`MOVE CORR WS-ORIGEM TO WS-DESTINO`